Object Oriented Programming (CSEG1044)
Unit 03 – Lecture 04 Notes
Java I/O Streams + FileReader/FileWriter

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Prerequisites **2**

3 Learning Outcomes **2**

4 Suggested Reading Order **2**

5 Lecture Overview **3**

6 Prerequisite Bridge **3**

7 Key Terms (Quick) **4**

8 Detailed Notes **4**

8.1 Stream model (big picture) 4
8.2 Byte streams vs character streams 4
8.3 Text file I/O with `FileReader`/`FileWriter` 4
8.4 Reading patterns (three common styles) 5
8.5 Exception-safe I/O: `try-with-resources` 5
8.6 Paths and a common beginner mistake 5
8.7 Overwrite vs append (writing) 6

9 Running Project Connection **6**

10 Common Mistakes and Confusions **6**

11 Worked Example (tutorial): read marks and write a report **6**

12 Demo / Observation Task **8**

13 Mini tutorial: copy a text file **8**

14 Practice Check (with brief solutions) **9**

15 Self-Study Practice (with hints) **9**

16 Summary (what to remember) **9**

17 Lab Alignment (recommended) **9**

18 Further Study **10**

19 Exit Questions (with sample answers) **10**

1 Lecture Snapshot

- **Unit focus:** Nested Classes, Exceptions, Multithreading, and I/O
- **Main new ideas:** Stream model (big picture), Byte streams vs character streams, Text file I/O with `FileReader`/`FileWriter`
- **Course thread:** Use Java I/O Streams + FileReader/FileWriter to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

2 Prerequisites

You should already know:

- basic exception handling (`try/catch`),
- loops and string trimming/parsing,
- how to run a Java program from an IDE or terminal.

3 Learning Outcomes

After this lecture, you should be able to:

- Explain stream model; choose byte vs character streams.
- Read/write text files with `FileReader`/`FileWriter` and buffering.
- Use `try-with-resources` for exception-safe I/O code.

4 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.

2. Study the central explanation in this order: *Stream model (big picture) → Byte streams vs character streams → Text file I/O with `FileReader`/`FileWriter` → Reading patterns (three common styles) → Exception-safe I/O: `try-with-resources`.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

5 Lecture Overview

In almost every real application you must read/write data:

- read configuration,
- read input files (CSV/text),
- write reports/logs,
- store results for later use.

Java provides a stream-based I/O model. In this lecture we focus on text files with `FileReader`, `FileWriter`, and buffering. We also address common beginner mistakes such as wrong paths, unclosed files, and exception confusion.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

6 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Exceptions**: File work is one of the most common places where exceptions matter. Main reference: `Unit 03 / Lecture 02 slides/notes`.
- **Strings and loops**: Reading and writing files still depends on iterating and representing text correctly. Main reference: `Unit 01 / Lecture 04 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

7 Key Terms (Quick)

- **Stream**: sequence of data flowing from a source to a destination.
- **Input stream** vs **output stream**: read vs write.
- **Byte stream**: reads/writes raw bytes (`InputStream`/`OutputStream`).
- **Character stream**: reads/writes characters (`Reader`/`Writer`).
- **Buffering**: collecting data in memory to reduce expensive I/O operations.
- **try-with-resources**: `try(...)` that auto-closes resources.

8 Detailed Notes

8.1 Stream model (big picture)

I/O in Java is organized as streams:

- A **source** (keyboard, file, network),
- A **stream object** (API you use),
- A **destination** (screen, file, network).

Streams are **one-directional**: you create an input stream to read and an output stream to write. You can also **chain** streams: for example, a file reader wrapped by a buffered reader.

8.2 Byte streams vs character streams

Byte streams (`InputStream`/`OutputStream`) are used for binary data:

- images, PDFs, audio/video, serialized binary formats.

Character streams (`Reader`/`Writer`) are used for text:

- plain text files, logs, reading lines of text.

Rule of thumb. Use `Reader/Writer` for text; use `InputStream/OutputStream` for binary.

8.3 Text file I/O with `FileReader`/`FileWriter`

Basic idea:

- `FileReader` reads characters from a file.
- `FileWriter` writes characters to a file.
- These can be wrapped in buffered versions for performance.

Important note (encoding). `FileReader/FileWriter` use the platform default charset. For most course examples this is fine, but in real projects you often choose a charset explicitly (UTF-8).

Buffering:

- `BufferedReader` provides `readLine()` and reads in chunks.
- `BufferedWriter` provides efficient writing and `newLine()`.

8.4 Reading patterns (three common styles)

1) Read character-by-character (rarely used in app code):

- simple idea, but slower and harder to manage.

2) Read line-by-line (most common for text files):

- use `BufferedReader.readLine()`.

3) Read tokens/CSV (usually needs parsing/splitting after reading lines):

- read line → split by delimiter → parse fields.

8.5 Exception-safe I/O: `try-with-resources`

File I/O can fail for many reasons: wrong path, missing file, permissions. Also, resources must be closed.

Use:

- `try (BufferedReader br = ...){...}` to auto-close even if exception occurs.

This reduces resource leaks and makes code cleaner.

8.6 Paths and a common beginner mistake

The JVM reads files relative to the "working directory" (where you run `java`). If the file is "not found":

- confirm the path,
- print the working directory (or use absolute path),
- ensure file extension and capitalization are correct.

Quick debug snippet (print working directory).

```
System.out.println(new java.io.File(".").getAbsolutePath());
```

Windows path gotcha (Java strings). In Java string literals, backslash is an escape character:

- wrong: `"D:\data\marks.txt"` (contains escape sequences)
- correct: `"D:\\data\\marks.txt"` or `"D:/data/marks.txt"`

8.7 Overwrite vs append (writing)

`FileWriter(path)` overwrites the file by default. To append:

```
new FileWriter("log.txt", true); // append=true
```

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

9 Running Project Connection

Use Java I/O Streams + FileReader/FileWriter to make Campus Resource Manager safer, more modular, and easier to recover when something goes wrong.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

10 Common Mistakes and Confusions

- Forgetting to close streams or readers.
- Using byte streams when character streams fit text better.
- Reporting vague file errors with no recovery hint.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

11 Worked Example (tutorial): read marks and write a report

Goal

Read integer marks from `marks.txt` (one mark per line), compute sum/avg/max, and write a report to `report.txt`.

Input file example

```
78
91
65
```

```
88
```

Blank lines are allowed; we will ignore them. If the file has no numeric rows, report the maximum as `N/A` instead of printing a fake value.

Code

```java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class MarksReport {
    public static void main(String[] args) {
        String inPath = "marks.txt";
        String outPath = "report.txt";

        int sum = 0, max = 0, count = 0;

        try (BufferedReader br = new BufferedReader(new FileReader(inPath))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty()) continue;

                int x = Integer.parseInt(line);
                if (count == 0 || x > max) max = x;
                sum += x;
                count++;
            }
        } catch (IOException e) {
            System.out.println("I/O error: " + e.getMessage());
            return;
        } catch (NumberFormatException e) {
            System.out.println("Invalid number in input file: " + e.getMessage());
            return;
        }

        double avg = (count == 0) ? 0.0 : (sum / (double) count);

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(outPath))) {
            bw.write("Count: " + count);
            bw.newLine();
            bw.write("Sum: " + sum);
            bw.newLine();
            bw.write("Max: " + (count == 0 ? "N/A" : max));
            bw.newLine();
            bw.write(String.format("Avg: %.2f", avg));
            bw.newLine();
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }
```

```
    }
}
```

What to notice

- `try-with-resources` closes readers/writers automatically.
- `BufferedReader.readLine()` makes line-by-line reading easy.
- We handle both I/O errors and invalid numbers separately (better messages).
- We use `String.format` for formatted average.

Common improvements (good practice)

- Validate mark range (0–100) and count invalid rows.
- Show a clearer message when the file is missing (path help).
- Write results in a CSV format if needed.

12 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Stream model (big picture)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

13 Mini tutorial: copy a text file

Copying is a great exercise because it uses both reading and writing.

```java
import java.io.*;

public class CopyTextFile {
    public static void main(String[] args) throws IOException {
        String src = "input.txt";
        String dst = "output.txt";

        try (BufferedReader br = new BufferedReader(new FileReader(src));
             BufferedWriter bw = new BufferedWriter(new FileWriter(dst))) {
```

```
            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }
        }
    }
}
```

14 Practice Check (with brief solutions)

1. Which should be used for text files: byte stream or character stream?

   Answer: character stream (`Reader/Writer`).

2. Why do we use buffering?

   Answer: it reduces the number of expensive disk I/O calls by reading/writing in chunks.

3. If a file is not found, name one common cause.

   Answer: wrong working directory / wrong path.

15 Self-Study Practice (with hints)

1. Write a program that copies a text file (read from one, write to another) using buffered streams. **Hint:** the copy tutorial above is the solution pattern.

2. Write a program that counts lines and words in a file. **Hint:** count lines with `readLine()` and count words with `split("\\s+")`.

3. Modify the worked example to ignore marks outside 0–100 and count how many were invalid. **Hint:** after parsing, check range and continue.

16 Summary (what to remember)

- Streams model: input vs output; chain streams for features like buffering.
- Use `Reader/Writer` for text and buffer with `BufferedReader/BufferedWriter`.
- Use `try-with-resources` for reliable closing.
- Most file bugs come from wrong paths/working directory: print it and verify.
- Handle `IOException` separately from parse/validation exceptions.

17 Lab Alignment (recommended)

- Exp-13: File Handling using I/O streams (based on this lecture)

18 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed April 1, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed April 1, 2026).

19 Exit Questions (with sample answers)

Q1. Why is `try-with-resources` preferred?

Sample answer: It automatically closes resources even if an exception occurs, preventing resource leaks and reducing boilerplate code.

Q2. When do you choose byte streams instead of character streams?

Sample answer: Use byte streams for binary data like images or PDFs. Use character streams for text data.

Q3. Why is try-with-resources preferred for file work?

Sample answer: It closes resources automatically and keeps the success path easier to read.